

EE-170L Computer Programming Lab
Laboratory Session 10

More About Functions in C++

Name:	Shahzad Gul
Semester:	2
Section:	B
Reg No:	0671
Lab:	10
Date:	Spring 2026

1. Objective

The objective of this lab session is to understand and apply advanced function concepts in C++ programming. Students will learn the difference between call by value and call by reference, understand how default parameters work, and implement functions with parameters and return values. By the end of this lab, students will be able to write modular programs using functions effectively.

Key learning outcomes include:

- Understanding function call by value and its behavior
- Understanding function call by reference and its advantages
- Using default values in function parameters
- Writing functions that accept parameters and return values
- Creating modular and reusable code using functions

2. Theory

2.1 Call by Value

Call by value is used when the called function does not need to modify the value of the caller's original variable. In this case, a copy of the variable is passed to the function. When the function changes its value, it has no effect on the original variable in the calling function. This is the default parameter passing mechanism in C++. It is safe because the original data remains unchanged.

Example of call by value:

```
void func(int i)
{
    i = i + 10; // Changes local copy only
}

int main()
{
    int i = 10;
    func(i); // Passes copy of i
    // i still equals 10 in main
}
```

2.2 Call by Reference

Call by reference means the compiler does not create a local copy of the variable. Instead, it passes the memory address of the variable. When operations are performed on a referenced variable, the changes affect the original variable directly. The **&** (ampersand) operator is used to indicate a reference parameter. This is useful when a function needs to modify the caller's variables or when passing large objects efficiently.

Example of call by reference:

```
void swap(int &x, int &y)
{
    int temp = x;
    x = y;
    y = temp; // Changes original variables
}

int main()
{
    int a = 5, b = 10;
    swap(a, b); // a and b are actually swapped
}
```

2.3 Default Values in Parameters

When declaring a function, we can specify a default value for each of the last parameters. This value is used if the corresponding argument is omitted when calling the function. To specify a default value, use the assignment operator in the function declaration. If a value is passed, the

default is ignored; if no value is passed, the default is used.

Example of default parameters:

```
int divide(int a = 8, int b = 2)
{
    return a / b;
}

// Function calls:
divide();          // Uses defaults: 8/2 = 4
divide(12);        // a=12, b=2: 12/2 = 6
divide(63, 7);     // a=63, b=7: 63/7 = 9
```

Comparison: Call by Value vs Call by Reference

Feature	Call by Value	Call by Reference
Copy Created	Yes, a copy of the argument is passed	No, the address is passed
Original Data	Remains unchanged	Can be modified by function
Memory Usage	Uses extra memory for copies	Memory efficient (no copies)
Syntax	void func(int x)	void func(int &x;)
Use Case	When original data must be protected	When function needs to modify original data

3. Lab Tasks

Task 1: Function with Parameters

Objective: Declare a function called "multiply" that takes two integer parameters: "num1" and "num2". Inside the function, calculate the product of the two numbers. Display the result within the function. Call the function from the main program, passing two numbers, to perform the multiplication.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 10 - Task 1: Function with Parameters
 * Description: Program that declares a function called "multiply" that takes
 *              two integer parameters and calculates their product.
 */

#include <iostream>
using namespace std;

// Function prototype
void multiply(int num1, int num2);

int main()
{
    // Declare two integer variables
    int a, b;

    // Prompt user to enter two numbers
    cout << "Enter first number: ";
    cin >> a;
    cout << "Enter second number: ";
    cin >> b;

    // Call the function with two arguments
    multiply(a, b);

    return 0;
}

// Function definition
void multiply(int num1, int num2)
{
    // Calculate the product of two numbers
    int product = num1 * num2;

    // Display the result within the function
    cout << "The product of " << num1 << " and " << num2
         << " is: " << product << endl;
```

```
}
```

Expected Output:

```
Enter first number: 5  
Enter second number: 4  
The product of 5 and 4 is: 20
```

Explanation: The program defines a function **multiply()** that accepts two integer parameters. The function calculates the product and displays it directly. This demonstrates a **void function** with parameters that performs an operation without returning a value. The function is called from **main()** with user-provided arguments, illustrating the call by value mechanism.

Task 2: Function with Return Value

Objective: Declare a function called "getSquare" that takes an integer parameter: "number". Inside the function, calculate the square of the number. Return the result. Call the function from the main program, passing a number, and display the returned value.

Source Code:

```
/*
 * Name: Shahzad Gul
 * Semester: 2
 * Section: B
 * Reg No: 0671
 * Lab: 10 - Task 2: Function with Return Value
 * Description: Program that declares a function called "getSquare" that
 *              takes an integer parameter and returns its square.
 */

#include <iostream>
using namespace std;

// Function prototype
int getSquare(int number);

int main()
{
    // Declare a variable
    int num;

    // Prompt user to enter a number
    cout << "Enter a number: ";
    cin >> num;

    // Call the function and store the returned value
    int result = getSquare(num);

    // Display the result
    cout << "The square of " << num << " is: " << result << endl;

    return 0;
}

// Function definition
int getSquare(int number)
{
    // Calculate the square of the number
    int square = number * number;

    // Return the result to the calling function
    return square;
}
```

Expected Output:

```
Enter a number: 7
The square of 7 is: 49
```

Explanation: The program defines a function **getSquare()** that accepts an integer parameter and returns its square. The return value is captured in a variable in **main()** and then displayed. This demonstrates a **value-returning function**, which is essential for creating reusable calculations that can be used in expressions and assignments.

4. Conclusion

In this lab session, we successfully explored advanced function concepts in C++. We learned the fundamental differences between call by value and call by reference, and how default parameters can make functions more flexible. The lab tasks demonstrated practical applications of function concepts, including:

- Writing functions that accept multiple parameters and perform calculations
- Creating value-returning functions for reusable computations
- Understanding the importance of function prototypes for proper compilation
- Applying modular programming principles to organize code effectively

Functions are the building blocks of structured programming. Understanding parameter passing mechanisms and return values is crucial for developing efficient, maintainable, and reusable code. These concepts form the foundation for object-oriented programming and advanced software development techniques.

5. References

- Deitel, P. & Deitel, H. (2017). *C++ How to Program* (10th ed.). Pearson.
- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
- UET Peshawar, EE-170L Computer Programming Lab Manual, Spring 2026.